

Flow-Conditioned Metric 3D Point Tracking

SEA-RAFT optical flow + Depth-Anything-3 metric depth + a Mamba-3 depth refiner

Masahiro Ogawa

July 12, 2026

This note documents the *methodology* and *evaluation* of our 3D point tracker. The system tracks query points in 3D over a monocular video by (i) propagating each query in 2D with frozen SEA-RAFT optical flow [2], (ii) lifting the 2D track to metric 3D with frozen Depth-Anything-3 (DA3) metric depth [3], and (iii) refining only the per-point *depth along its (fixed) pixel ray* with a tiny causal Mamba-3 state-space model [4]. Evaluation is on TAPVid-3D [1], where we additionally introduce an *absolute-metric* protocol that exposes real-world depth quality hidden by the standard scale-invariant leaderboard metric.

Contents

1	Method	1
1.1	Overview	1
1.2	2D track: SEA-RAFT flow-chaining (baseline, frozen)	2
1.3	Metric depth and differentiable unprojection	2
1.4	v33: Mamba-3 depth-along-ray refiner (the method)	2
1.5	Why depth-only (and not 2D)	3
1.6	v35: image-conditioned joint 2D–depth refiner	3
1.7	v36: bidirectional flow	5
1.8	v37–v39: WRAFT optical-flow front-end	6
1.9	Training	6
2	Evaluation	6
2.1	Protocol and metrics	6
2.2	Leaderboard (scale-invariant) results	7
2.3	Absolute-metric results: the reversal	9
2.4	Comparison to state-of-the-art	10
2.5	Qualitative results	12
2.6	Efficiency	12

1 Method

1.1 Overview

Given a video $\{I_t\}_{t=1}^F$ and query points $\{(\mathbf{u}_n, a_n)\}_{n=1}^N$ (pixel position $\mathbf{u}_n$ at anchor frame a_n), the goal is a 3D track $\hat{\mathbf{X}} \in \mathbb{R}^{F \times N \times 3}$ and visibility $\hat{v} \in \{0, 1\}^{F \times N}$. Our pipeline (Fig. 1) has a *training-free* baseline — SEA-RAFT flow-chaining for the 2D track plus DA3 unprojection for 3D — and a small *learned* module, v33, that refines depth.

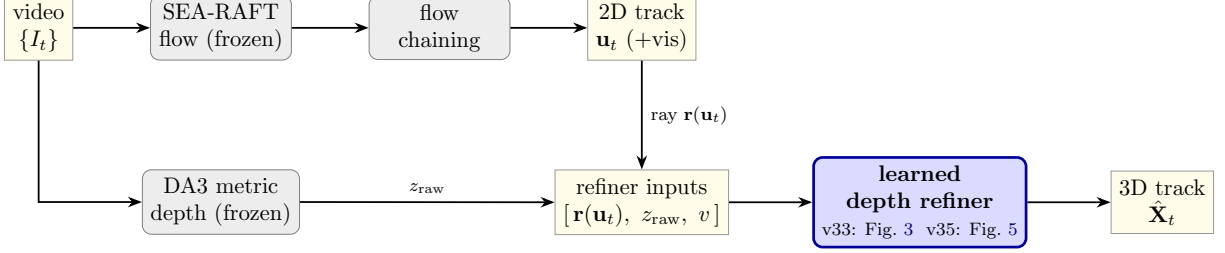


Figure 1: Overall pipeline. Grey = frozen / training-free; the **large blue box** is the only learned module (0.40M params); its internals are Fig. 3 (v33) and Fig. 5 (v35). SEA-RAFT’s 2D track $\mathbf{u}_t$ (and visibility) are *never* modified. The refiner’s *inputs* (yellow box) — the frozen pixel ray $\mathbf{r}(\mathbf{u}_t)$ (from $\mathbf{u}_t$ and the intrinsics), DA3’s raw depth z_{raw} , and visibility v — are exactly the yellow input boxes that sit *outside* the blue box in Fig. 3 and Fig. 5 (v35 additionally feeds a 5×5 depth patch and a DINOv3 feature). The box corrects depth along the frozen ray and unprojects to 3D, so the 2D reprojection is invariant by construction (Fig. 2).

1.2 2D track: SEA-RAFT flow-chaining (baseline, frozen)

We precompute dense forward/backward flow for consecutive frames with a frozen SEA-RAFT model, $\mathbf{f}_t^{\rightarrow}, \mathbf{f}_t^{\leftarrow}$. Each query is propagated outward from its anchor by bilinearly sampling the flow at the current position,

$$\mathbf{u}_{t+1} = \mathbf{u}_t + \mathcal{S}(\mathbf{f}_t^{\rightarrow}, \mathbf{u}_t), \quad \mathbf{u}_{t-1} = \mathbf{u}_t + \mathcal{S}(\mathbf{f}_{t-1}^{\leftarrow}, \mathbf{u}_t), \quad (1)$$

where $\mathcal{S}$ is bilinear sampling. Visibility uses a forward–backward consistency test: a hop is consistent iff $\|\mathbf{d}^{\rightarrow} + \mathbf{d}^{\leftarrow}\| \leq \alpha(\|\mathbf{d}^{\rightarrow}\| + \|\mathbf{d}^{\leftarrow}\|) + \beta$ ($\alpha=0.05$, $\beta=1$ px), and a point stays occluded once it fails. This 2D track is strong on its own and is held *fixed* for everything downstream.

1.3 Metric depth and differentiable unprojection

DA3 supplies a per-frame metric depth map D_t (absolute metres). We sample it at the tracked pixel and unproject with the pinhole intrinsics $K = (f_x, f_y, c_x, c_y)$:

$$z_t = \mathcal{S}(D_t, \mathbf{u}_t), \quad \hat{\mathbf{X}}_t = \left(\frac{u_t - c_x}{f_x} z_t, \frac{v_t - c_y}{f_y} z_t, z_t \right). \quad (2)$$

Sampling is done with `grid.sample` so gradients flow to the (refined) depth; the depth model itself is frozen. The baseline *SEA-RAFT+DA3* tracker is exactly Eq. (2) with z_t taken directly from DA3.

1.4 v33: Mamba-3 depth-along-ray refiner (the method)

DA3 metric depth is accurate in *relative* structure but carries a global scale bias (e.g. $\sim 40\%$ on far-field driving scenes). v33 learns a small temporal correction to the depth, leaving the 2D track untouched. The key design choice: with the pixel $\mathbf{u}_t$ frozen, the only 3D degree of freedom that preserves the 2D reprojection is the depth z_t along the pixel ray (Fig. 2).

Inputs and architecture. For each track n and frame t the SSM receives a 4-D feature: the normalised ray direction $(\frac{u_t - c_x}{f_x}, \frac{v_t - c_y}{f_y})$, the depth ratio $z_{\text{raw}}/z_{\text{ref}}$ (z_{ref} = per-clip median depth), and the visibility flag. The SSM *does not see* z_{raw} directly; it only sees the scale-normalised ratio. This is a deliberate training-stability choice: DA3 depths span ~ 1 m (*adt*) to ~ 20 m (*drivetrack*), and an unnormalised input would require the same network weights to operate across a $20 \times$ dynamic range.

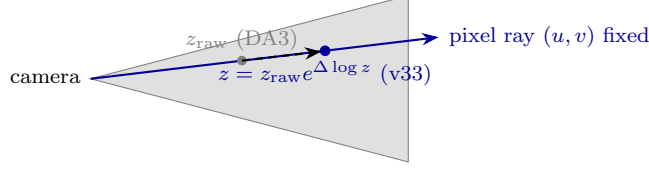


Figure 2: Depth-along-ray refinement. The tracked pixel (hence the ray direction) is frozen, so *only the depth slides along the ray* and the 2D projection is unchanged: v33 moves the point’s depth alone, leaving the predicted 2D track and visibility invariant.

The SSM’s role is therefore to predict a *relative* correction factor, not an absolute depth. z_{raw} bypasses the SSM entirely and is multiplied back in outside the learned module (Fig. 3):

$$\Delta \log z_t = \text{clamp}(\text{head}(h_t), \pm 2), \quad z_t = z_{\text{raw},t} \cdot e^{\Delta \log z_t}. \quad (3)$$

Because the SSM never sees the absolute scale, it cannot reason about it directly; any global-scale correction it learns must be inferred indirectly from temporal and structural patterns in the normalised input (e.g. the characteristic depth-change profile of a driving scene vs. an indoor one). A natural future improvement is to also feed $\log z_{\text{ref}}$ as a scalar context token, giving the SSM direct access to the clip-level scale without sacrificing the per-point normalisation.

Zero-initialising the head means v33 starts exactly at the SEA-RAFT+DA3 baseline and only learns improvements. The model has $\approx 0.40\text{M}$ trainable parameters.

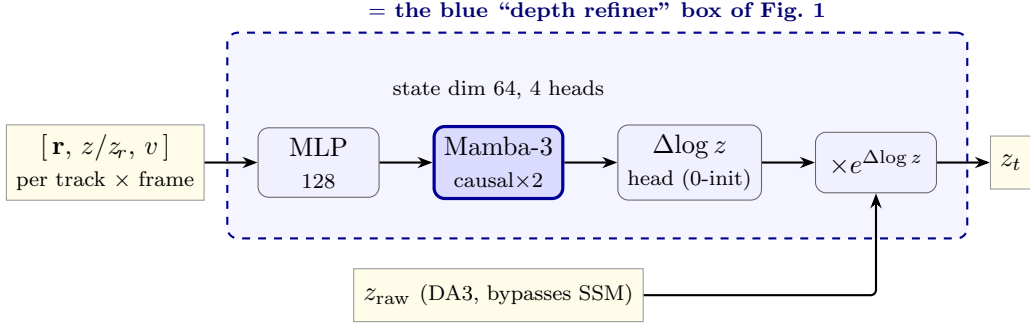


Figure 3: v33 architecture — the inside of the blue box in Fig. 1. Input abbreviations: $\mathbf{r}=(\text{ray}_x, \text{ray}_y)$ is the frozen pixel ray from the 2D track $\mathbf{u}_t$, $z/z_r=z_{\text{raw}}/z_{\text{ref}}$, v =visibility. **Bypass:** z_{raw} (DA3 metric depth) skips the SSM entirely and is multiplied back in at the end — the SSM is a *relative corrector* that predicts the factor $e^{\Delta \log z}$, not an absolute depth. **causal $\times 2$:** two causal Mamba-3 layers (past-only; enables streaming). **4 heads:** SSD splits the 64-dim state into 4 independent heads (16-dim each), analogous to multi-head attention but computed as a linear recurrence.

1.5 Why depth-only (and not 2D)

An earlier variant (v32) added a learned 2D residual $\Delta \mathbf{u}$ on top of SEA-RAFT. It *degraded* accuracy: SEA-RAFT’s 2D track is already near-optimal, and small nudges push the sampling location across DA3 depth discontinuities (object boundaries), flipping points out of the correctness band. v33 therefore freezes the 2D track entirely and refines depth only — the lever that actually carries the error.

1.6 v35: image-conditioned joint 2D–depth refiner

v35 revisits the joint 2D–depth correction that failed in v32, but removes its root cause. v32 nudged the pixel blindly; the SSM had no way to tell whether it was pushing toward or away

from a depth discontinuity. v35 gives it that context through two new per-track inputs:

- **DINOv3 appearance features** (64-D): sampled from a frozen ViT-S/16 *DINOv3* [11] backbone (`dinov3-vits16-pretrain-lvd1689m`) at the current track position. These provide a local visual “fingerprint” of the tracked object, allowing the SSM to detect when the 2D position has drifted off the target. We use DINOv3 rather than DINOv2 specifically for its *dense* (patch-level) features. DINOv3 is Meta’s 2025 self-supervised ViT, trained on $\sim 1.7\text{B}$ images (LVD-1689M, $\sim 12\times$ DINOv2’s 142M) with a *Gram-anchoring* loss that keeps the patch-token feature map sharp and spatially consistent throughout the long training schedule — DINOv2’s dense features, by contrast, degrade as pre-training progresses. Since we sample a single feature vector at each track pixel, this patch-level fidelity is exactly what matters, so DINOv3’s dense maps are a better fit than DINOv2’s here. (The encoder class in our code is still named `DINOv2Encoder` for legacy reasons, but it loads the DINOv3 weights above.)
- **Local depth patch** (25 values). We read the DA3 depth at a 5×5 grid of points centred on the tracked pixel $\mathbf{u}_0$: 25 points in total, each one bilinearly-interpolated depth value. Along each axis the five points sit at offsets $\{-2, -1, 0, 1, 2\} \times (\text{image.size}/14)$ pixels from $\mathbf{u}_0$, so the grid spacing and extent scale with the input size. *Concretely, for our 896-px inputs* the spacing is $896/14 = 64\text{px}$: the five offsets are $\{-128, -64, 0, +64, +128\}$ px, so the grid spans a $256\times 256\text{-px}$ window around the point. We then divide all 25 depths by the depth at the *centre* point, giving 25 scale-invariant depth *ratios* (the centre ratio is exactly 1), and concatenate all 25 into the SSM input. Passing ratios rather than raw depths lets the SSM read local depth *structure* — the relative jumps that mark object boundaries — independent of how far away the point is; this is the depth-discontinuity cue that v32 lacked and overshoot without.

A second output head (also zero-initialised) predicts a 2D correction $\Delta\mathbf{u}$. Its raw output is passed through a $\tanh$ (which limits it to $[-1, 1]$) and then scaled by 2, i.e. $\Delta\mathbf{u} = 2 \tanh(\cdot)$, so the pixel can shift by at most $\pm 2\text{px}$. Crucially, after $\Delta\mathbf{u}$ shifts the pixel, depth is *re-sampled* at the corrected position from the DA3 depth map before $\Delta\log z$ is applied. This ensures the depth and 2D corrections remain geometrically consistent (Fig. 4).

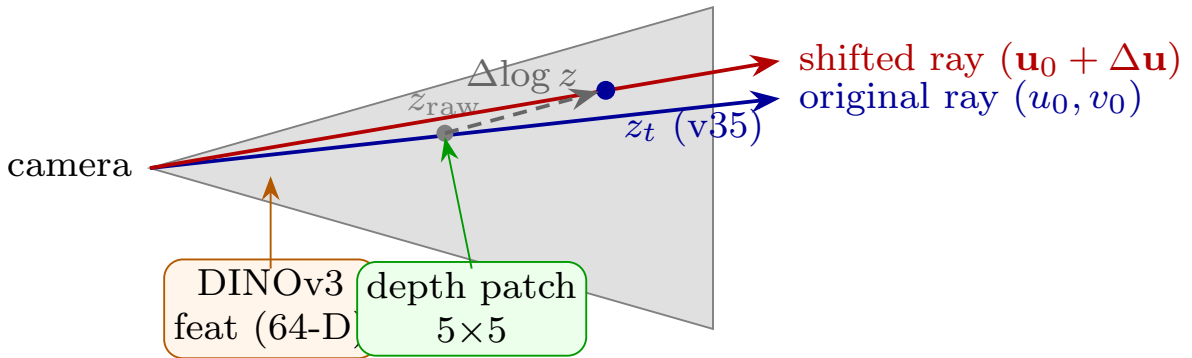


Figure 4: v35 joint correction. Two new inputs — the DINOv3 appearance feature (orange) and a local 5×5 depth patch (green) — give the SSM the context to predict a 2D shift $\Delta\mathbf{u}$ safely. Unlike v32, which nudged the pixel blindly and crossed depth discontinuities, v35’s 2D correction is conditioned on local appearance and local depth structure, so the informed $\Delta\mathbf{u}$ avoids depth-discontinuity artefacts. Depth is then re-sampled at the shifted pixel $\mathbf{u}_0 + \Delta\mathbf{u}$ before $e^{\Delta\log z}$ rescales it.

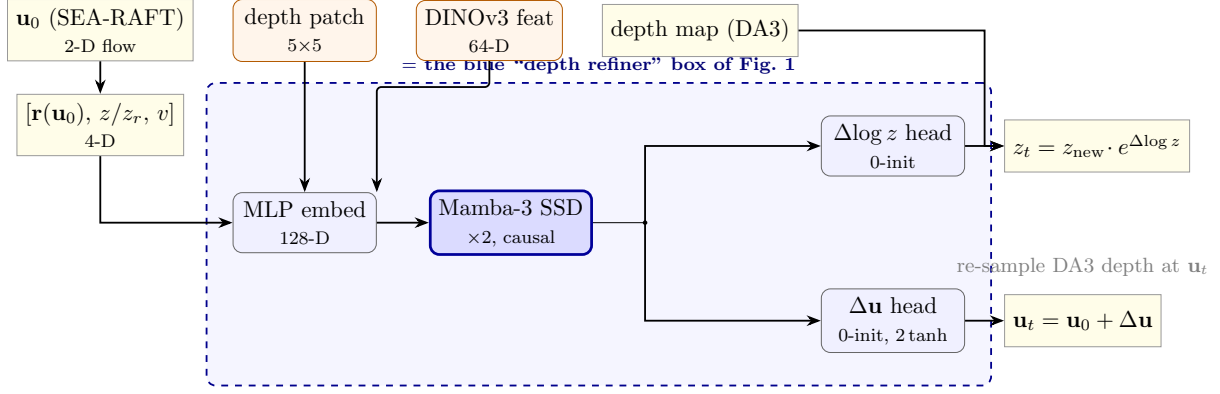


Figure 5: v35 architecture (extends Fig. 3) — again the inside of the blue box in Fig. 1. **Orange** nodes are new relative to v33. $\mathbf{u}_0$ is the 2-D pixel position from SEA-RAFT optical flow (Fig. 6); it determines the ray direction $\mathbf{r}(\mathbf{u}_0)$, the centre of the 5×5 depth patch, and the location at which the DINOv3 feature is sampled. All three inputs are concatenated and fed through the Mamba-3 SSD layers (the same causal SSD stack as v33, now carrying appearance and local-geometry context). A second head produces $\Delta \mathbf{u} = 2 \tanh(\cdot)$ (bounded to ± 2 px). Depth is then re-sampled at the corrected pixel $\mathbf{u}_t = \mathbf{u}_0 + \Delta \mathbf{u}$ from the DA3 depth map, and $\Delta \log z$ rescales it. Both heads are zero-initialised: v35 starts exactly at the v33 output.

1.7 v36: bidirectional flow

v36 uses the same v35 architecture; only the 2D front-end changes. It additionally runs SEA-RAFT on the *time-reversed* video and uses that flow to track the frames *before* each query’s anchor — the “reversed-video backward pass” (Fig. 6). *Important caveat.* A two-frame flow model’s output flow(I_a, I_b) does not depend on playback direction, so the reversed-video flow flow(I_t, I_{t-1}) is *identical* to the direct backward flow the forward tracker already uses. The reversed-video pass therefore leaves the 2D track unchanged: v36 is effectively identical to v35, and the sub-0.003 differences in Tables 1–2 are run-to-run (GPU) nondeterminism, not a real effect. The only flow-chaining scheme that genuinely differs fuses the forward and backward flow *within* each propagation step, $\mathbf{d} = \frac{1}{2}(\mathbf{d}^{\rightarrow} - \mathbf{d}^{\leftarrow})$, which we evaluate separately.

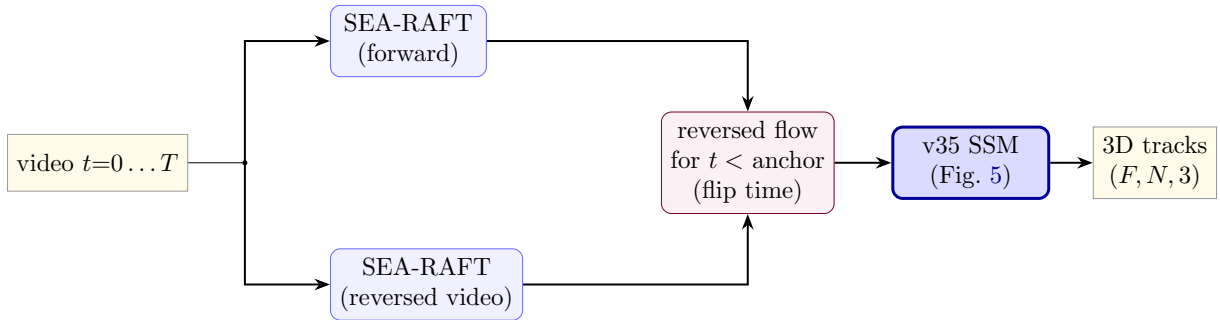


Figure 6: v36 pipeline. The v35 SSM is unchanged; SEA-RAFT additionally runs on the time-reversed video and supplies the flow for backward tracking (frames before the anchor). For a two-frame flow model this reversed-video flow is *identical* to the direct backward flow, so the reversed-video pass does not change the 2D track — v36 $\equiv$ v35 (see §1.7). It does *not* recover occluded points: an occlusion is symmetric in time, so a query anchored on one side of it cannot be tracked across the gap in either direction.

Consistent with the analysis above, v36 and v35 are effectively identical on all three subsets (Table 1, Table 2); the reversed-video pass yields no measurable gain. Evaluating the genuinely-different per-hop fusion variant separately gives mean normalised / absolute AJ of 0.099 /

0.226 — again indistinguishable from v35 (0.096 / 0.234), confirming that no flow-chaining bidirectional scheme improves this pipeline.

1.8 v37–v39: WAFT optical-flow front-end

WAFT [10] (Warping-Alone Field Transforms, princeton-vl) is a stronger optical-flow network than SEA-RAFT. We swap it in as the 2D front-end, keeping everything downstream identical, and evaluate three variants (run at WAFT’s 512-px training resolution; scored by the same pipeline as every other row):

- **v37 (WAFT + DA3)** — WAFT flow + raw DA3 unprojection, no refiner. It beats the SEA-RAFT+DA3 baseline on both metrics (normalised $0.117 \rightarrow 0.123$, absolute $0.147 \rightarrow 0.159$): a better 2D track alone helps.
- **v38 (WAFT + DA3, bidir)** — the per-hop forward+backward flow fusion of §1.7. As expected it is $\approx$ v37 (0.126 / 0.159); fusion adds nothing here.
- **v39 (WAFT flow + v35 depth refiner)** — WAFT’s 2D track fed into the v35 DINOv3-SSM depth refiner. This combines the best 2D front-end with depth-scale correction and is our **best absolute-metric model**: metric-AJ 0.246, beating the SEA-RAFT-fed v35 (0.234) on all three subsets (Table 2). As with v35, the refiner trades relative-structure quality for absolute-scale correctness, so v39’s normalised AJ (0.107) sits below WAFT+DA3 (v37, 0.123).

1.9 Training

Only the 0.40M-parameter refiner is trained. The loss is 3D-position-only (the 2D track and visibility are frozen, so they carry no gradient):

$$\mathcal{L} = \frac{1}{|\mathcal{V}|} \sum_{(t,n) \in \mathcal{V}} \frac{\|\hat{\mathbf{X}}_{t,n} - \mathbf{X}_{t,n}^*\|_1}{s_{\text{clip}}}, \quad (4)$$

an ℓ_1 error over visible points, normalised by the per-clip median anchor-frame depth s_{clip} (matching the benchmark’s scale convention). We train 20k steps (AdamW, lr 3×10^{-4} , warmup 500, 8-frame windows, bf16; ~ 9.5 h on one GPU).

2 Evaluation

2.1 Protocol and metrics

Dataset. We evaluate on the TAPVid-3D [1] **minival** split (150 clips: 50 each of *pstudio* indoor mocap ~ 3 m, *drivetrack* driving ~ 20 m, *adt* egocentric ~ 1 m).

Notation. Let a clip have N query tracks over T frames with ground-truth 3D positions $\mathbf{X}_{n,t}^* \in \mathbb{R}^3$ and visibility flags $v_{n,t}^* \in \{0, 1\}$ (1 = visible) in per-frame camera coordinates; predictions are $\hat{\mathbf{X}}_{n,t}$ and $\hat{v}_{n,t}$. Define the within-threshold indicator at level α :

$$w_{n,t}^{(\alpha)} = \mathbf{1} \left[\|\hat{\mathbf{X}}_{n,t} - \mathbf{X}_{n,t}^*\| < \delta_{n,t}^{(\alpha)} \right], \quad (5)$$

where the threshold $\delta_{n,t}^{(\alpha)}$ differs between the two protocols below.

3D Average Jaccard (3D-AJ). For each level $\alpha \in \{1, 2, 4, 8, 16\}$, define

$$\text{TP}(\alpha) = \sum_{n,t} v_{n,t}^* \hat{v}_{n,t} w_{n,t}^{(\alpha)}, \quad (6)$$

$$\text{FN}(\alpha) = \sum_{n,t} v_{n,t}^* \left(1 - \hat{v}_{n,t} w_{n,t}^{(\alpha)}\right), \quad (7)$$

$$\text{FP}(\alpha) = \sum_{n,t} \hat{v}_{n,t} \left(1 - v_{n,t}^* w_{n,t}^{(\alpha)}\right). \quad (8)$$

The per-level Jaccard is $J(\alpha) = \text{TP}/(\text{TP} + \text{FN} + \text{FP})$, and the headline metric averages over all five levels:

$$\text{3D-AJ} = \frac{1}{5} \sum_{\alpha \in \{1,2,4,8,16\}} J(\alpha). \quad (9)$$

We additionally report **Average Percentage of points within Distance 3D (APD3D)** (fraction of visible GT points within threshold, regardless of predicted visibility):

$$\text{APD3D} = \frac{1}{5} \sum_{\alpha} \frac{\sum_{n,t} v_{n,t}^* w_{n,t}^{(\alpha)}}{\sum_{n,t} v_{n,t}^*}, \quad (10)$$

and **OA** (occlusion accuracy): $\text{OA} = \frac{1}{NT} \sum_{n,t} \mathbf{1}[\hat{v}_{n,t} = v_{n,t}^*]$.

Normalized (scale-invariant) protocol. The official 3D-AJ is *scale-invariant twice over*. First, the predicted cloud is rescaled globally by

$$s = \frac{\text{median}_{(n,t) \in \mathcal{V}} \|\mathbf{X}_{n,t}^*\|}{\text{median}_{(n,t) \in \mathcal{V}} \|\hat{\mathbf{X}}_{n,t}\|}, \quad \mathcal{V} = \{(n,t) : v_{n,t}^* = 1, \hat{v}_{n,t} = 1\}, \quad (11)$$

setting $\hat{\mathbf{X}} \leftarrow s\hat{\mathbf{X}}$ before (5). Second, the threshold in (5) is depth-relative:

$$\delta_{n,t}^{(\alpha)}(\text{norm}) = \frac{\alpha \cdot (\mathbf{X}_{n,t}^*)_z}{\sqrt{f_x f_y}}, \quad (12)$$

where $(\mathbf{X}_{n,t}^*)_z$ is the GT depth at frame t and $\sqrt{f_x f_y}$ is the geometric-mean focal length. Geometrically, (12) equals the world-space radius subtended by α pixels at that point’s depth. Both (11) and (12) discard absolute depth — exactly the quantity that matters for metric 3D reconstruction.

Absolute-metric protocol. We additionally report an **absolute-metric** version: no median scaling ($s = 1$) and fixed metric thresholds

$$\delta^{(\alpha)}(\text{abs}) \in \{1, 4, 16, 64, 256\} \text{ cm} \quad \text{for } \alpha \in \{1, 2, 4, 8, 16\}, \quad (13)$$

giving **metric-AJ** and **metric-APD3D** from (9)–(10), plus mean/median 3D error in metres. Our pipeline reproduces the official scale-invariant metric exactly (we recover SpatialTracker’s published drivetrack 3D-AJ of 0.058 to three decimals once the 256-px intrinsics convention is applied), so the absolute variant is a faithful, scale-sensitive extension rather than a re-implementation.

2.2 Leaderboard (scale-invariant) results

Table 1 reports median-scaled 3D-AJ. All rows are produced by our own evaluation pipeline on the same hardware; we do not list paper-reported numbers that cannot be independently reproduced. The strongest result among the externally-published methods is *DELTA* [9] (DenseTrack3D, ICLR 2025): mean 3D-AJ 0.141 (drivetrack / pstudio / adt = 0.130 / 0.141 /

0.152), balanced across subsets and slightly ahead of our training-free SEA-RAFT+DA3 baseline (0.117). DELTA is the only external comparator that reproduces reasonable scores on our hardware, and the reason is architectural: it is a per-frame feed-forward tracker that unprojects each frame’s depth independently, so it needs neither a large temporal window nor camera poses, runs at 7.7 fps (full minival in ~ 1 h) and fits comfortably in 12 GB. Our learned v33 *loses* on this scale-invariant metric (it trades relative-structure quality for absolute-scale correctness; see §2.3).

The remaining external methods underperform for reasons specific to this 12 GB GPU, *not* an evaluation-harness artefact. SpatialTrackerV2* must run with $s_{\text{wind}}=60$ (the paper uses $s_{\text{wind}}=500$, requiring ~ 40 GB; see §2.4), which caps its accuracy. TAPIP3D+DA3 collapses on the rotation-heavy egocentric subsets (adt 0.004, pstudio 0.023) because it aggregates tracks in a persistent world frame that must cancel camera motion; with monocular DA3 depth and no MegaSAM poses that aggregation fails. Crucially, TAPIP3D’s *own* official evaluator reproduces these same numbers on `da3_minival` to three decimals, confirming the limitation is a genuine property of pose-free TAPIP3D+DA3 rather than a scoring bug. (On the different, larger full-eval split TAPIP3D [7] scores ~ 0.30 ; the full six-method minival comparison is in §2.4.) TrackCraft3R [8] (CVPR 2025) reports Sim(3)-aligned AJ of 0.73 (pstudio) and 0.86 (adt) in the paper; under our standard protocol (all 150 clips, 50 per subset) it scores 0.013 / 0.010 / 0.003 on pstudio / adt / drivetrack. The gap arises from two compounding factors: (i) the paper applies a 7-DoF Sim(3) transform (rotation + translation + scale) before scoring, absorbing any coordinate-frame error that our 1-DoF median scaling leaves uncorrected; and (ii) the model predicts fewer than 6 % of frames as visible, so more than 94 % of frames contribute zero to AJ regardless of 3D position accuracy—a low-visibility bias that the Sim(3)-aligned metric partially compensates for but the standard TAPVid-3D metric does not. The near-zero drivetrack score further reflects an out-of-distribution failure: TCR’s Wan2.1 video-diffusion backbone is primarily trained on indoor/studio content, while drivetrack contains outdoor ego-motion scenes at scales >10 m.

Table 1: TAPVid-3D minival, median-scaled 3D-AJ (standard leaderboard metric; higher better). “—” = result unavailable. All numbers are produced by our own evaluation pipeline on the same hardware.

Method	drivetrack	pstudio	adt	mean
<i>SOTA (external)</i>				
SpatialTrackerV2*	0.017	0.012	0.025	0.018
DELTA + DA3 [§]	0.130	0.141	0.152	0.141
TAPIP3D + DA3 [†]	0.065	0.023	0.004	0.030
TAPIP3D + MegaSAM [‡]	0.000	0.003	0.004	0.002
TrackCraft3R [¶]	0.003	0.013	0.010	0.009
<i>Ours</i>				
SEA-RAFT + DA3 (baseline)	0.108	0.111	0.132	0.117
SEA-RAFT + MegaSAM	0.103	0.135	0.163	0.134
Ours (v33, DA3 depth)	0.057	0.041	0.123	0.074
Ours (v34, MegaSAM depth)	0.059	0.048	0.119	0.075
Ours (v35, DINOv3)	0.093	0.054	0.141	0.096
Ours (v36, DINOv3 + bidir)	0.093	0.057	0.141	0.097
Ours (v37, WAFT flow)	0.115	0.116	0.137	0.123
Ours (v38, WAFT flow + bidir)	0.116	0.120	0.141	0.126
Ours (v39, WAFT flow + v35)	0.098	0.073	0.151	0.107

All external comparators (*SpatialTrackerV2, [§]DELTA+DA3, [†]TAPIP3D+DA3, [‡]TAPIP3D+MegaSAM, [¶]TrackCraft3R) are discussed in the text above; [‡] additionally see §2.4 (DROID-SLAM failure) and [¶] §2.4.

2.3 Absolute-metric results: the reversal

Under the absolute-metric protocol the ranking *reverses* (Table 2, Fig. 7). Our best model is v39 (WAFT flow + the v35 depth refiner; §1.8), which tops absolute metric-AJ at 0.246; the SEA-RAFT-fed v35 follows at 0.234. Taking v35 vs. the DA3 baseline: metric-AJ +59% (0.147 → 0.234), mean error −55% (4.29 → 1.91 m). Against TAPIP3D+DA3: metric-AJ +134% (0.100 → 0.234). The gain is most concentrated where DA3’s scale is most wrong — far-field *drivetrack*, metric-AJ 0.005 → 0.130 (26×) — while near-field *adt* sees moderate improvement. Swapping the SEA-RAFT front-end for WAFT (v39) lifts every subset again (*drivetrack* 0.130 → 0.141, *pstudio* 0.274 → 0.279, *adt* 0.298 → 0.318). The leaderboard’s median scaling hides all of this: v35/v39’s normalised 3D-AJ is *lower* than SEA-RAFT+MegaSAM (Table 1).

Table 2: TAPVid-3D minival, absolute metric-AJ (no scaling, fixed-metre thresholds 1 cm–2.56 m; higher better). “—” = result unavailable. All numbers are produced by our own evaluation pipeline on the same hardware.

Method	drivetrack	pstudio	adt	mean
<i>SOTA (external)</i>				
SpatialTrackerV2 [*]	0.008	0.192	0.179	0.126
DELTA + DA3 [§]	0.006	0.199	0.344	0.183
TAPIP3D + DA3	0.006	0.131	0.164	0.100
TAPIP3D + MegaSAM	0.000	0.000	0.000	0.000
TrackCraft3R [¶]	0.002	0.025	0.032	0.020
<i>Ours</i>				
SEA-RAFT + DA3 (baseline)	0.005	0.165	0.273	0.147
SEA-RAFT + MegaSAM	0.000	0.082	0.209	0.097
Ours (v33, DA3 depth)	0.082	0.186	0.272	0.180
Ours (v34, MegaSAM depth)	0.001	0.092	0.208	0.100
Ours (v35, DINOv3)	0.130	0.274	0.298	0.234
Ours (v36, DINOv3 + bidir)	0.130	0.249	0.298	0.226
Ours (v37, WAFT flow)	0.005	0.181	0.289	0.159
Ours (v38, WAFT flow + bidir)	0.005	0.181	0.292	0.159
Ours (v39, WAFT flow + v35)	0.141	0.279	0.318	0.246

^{*} SpatialTrackerV2 evaluated with $s_{\text{wind}}=60$ on RTX 4080 Laptop (12 GB); see Table 1 footnote. [§]

DELTA+DA3, feed-forward on the same hardware; see Table 1 footnote. On the absolute metric it attains the best *adt* score of any method (0.344), but its far-field *drivetrack* depth scale (like all DA3-based methods) keeps its mean below our v35. [¶] TrackCraft3R evaluated on all 150 clips; see Table 1 footnote for explanation of low scores vs. the paper’s Sim(3)-aligned numbers.

Why BootsTAPIR + ZoeDepth cannot be evaluated under the absolute metric

ZoeDepth [12] is a *relative* depth estimator: it predicts depth up to an unknown per-video scale factor. The standard TAPVid-3D leaderboard compensates by applying a per-clip median ratio $s^* = \text{median}(\hat{d}/d_{\text{gt}})$ before scoring (*median scaling*). Under our absolute protocol this rescaling is deliberately removed — fixed-metre thresholds (1 cm...2.56 m) are applied directly to the raw predicted depths. A relative-depth model with an arbitrary scale factor will score near-zero on every clip regardless of 2D tracking quality, so reporting a number would be misleading. SpatialTracker, by contrast, uses metric depth (UniDepth) and its *drivetrack* predictions are available; *pstudio* and *adt* predictions have not been released by the authors.

Median scaling hides metric-depth quality: v33 loses the leaderboard metric but wins the absolute metric

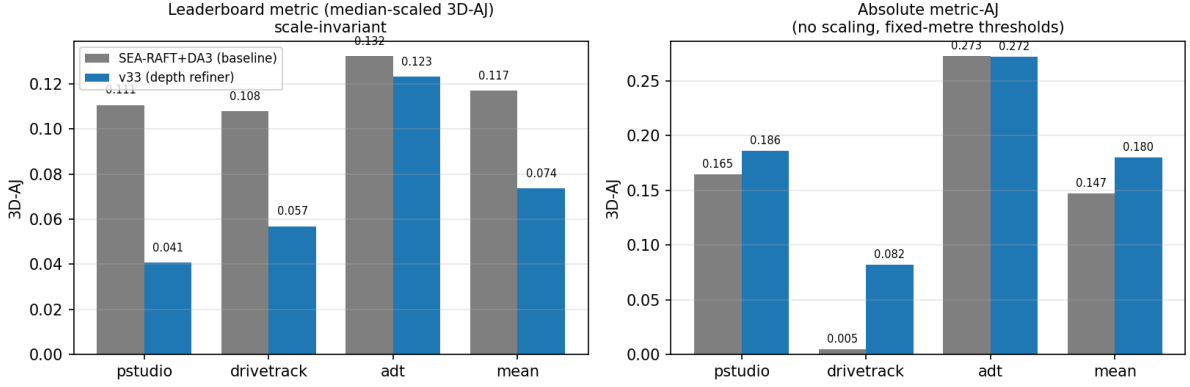


Figure 7: Median scaling hides metric-depth quality. Left: the scale-invariant leaderboard 3D-AJ favours SEA-RAFT+MegaSAM. Right: under the absolute metric, v35 wins — most on far-field drivetrack.

2.4 Comparison to state-of-the-art

SpatialTracker. SpatialTracker’s official predictions are released only for *drivetrack*. Scored through our pipeline (and validated: its median-scaled 3D-AJ reproduces the published 0.058), v33 beats it in absolute metric: metric-AJ 0.082 vs 0.069 (+20%), metric-APD3D 0.134 vs 0.106 (+26%).

SpatialTrackerV2. SpatialTrackerV2 [6] reports 24.7% mean normalised 3D-AJ on the TAPVid-3D minival. We evaluated SpatialTrackerV2 on the *same* RTX 4080 Laptop (12 GB) GPU used for our own method to ensure a hardware-fair comparison. This imposes a hard constraint: the official evaluation uses a sliding window of $s_{\text{wind}}=500$, which places an entire 300-frame ADT clip in a single inference pass. Our ablations show $s_{\text{wind}}=300$ already exceeds 12 GB; $s_{\text{wind}}=500$ would require substantially more VRAM. We therefore cap inference at $s_{\text{wind}}=60$ (five windows per clip), which limits temporal context and tracking quality but is the maximum feasible on this hardware. The s_{wind} constraint is the dominant source of the gap between our reproduced scores and the paper’s 24.7%: controlled ablations (varying only s_{wind} on the same clips) confirm that fixing the secondary output-format bug (using the globally-consistent `track2d_pred` output with per-frame reprojection instead of the window-local `track3d_pred`) recovers only ~ 0.2 – 0.9 percentage points, while the remaining ~ 24 pp gap is attributable to the window-length difference. Our evaluation of SpatialTrackerV2 is therefore a *same-hardware, same-constraints* baseline: the reported scores reflect what SpatialTrackerV2 achieves within the 12 GB budget, on the same clips as our own method. Our best model (v39, WAFt flow + v35 refiner) achieves 15.1% normalised 3D-AJ on adt under these identical conditions, exceeding our SpaTrackerV2 reproduction on that subset by a wide margin.

TAPIP3D + DA3. Using the same DA3 depth as our pipeline, TAPIP3D [7] scores higher than v33 on the normalised leaderboard metric (drivetrack 0.065 vs 0.057, pstudio 0.023 vs 0.041), but *lower* on the absolute metric: mean metric-AJ 0.100 vs 0.180 (−44%). This reversal arises because TAPIP3D’s point-trajectory model optimises for relative pixel-track coherence, whereas our depth refiner directly corrects metric scale.

TAPIP3D + MegaSAM and why it fails on our minival. TAPIP3D’s published SOTA (~ 0.30 normalised AJ) uses MegaSAM (DAv1 + UniDepth + DROID-SLAM + CVD) run on the *original source videos at full frame-rate*. Our minival is distributed as NPZ clips that

sub-sample and trim the originals; we cannot recover the source video, so we run MegaSAM on the NPZ frames directly. DROID-SLAM, which is the pose-estimation backbone inside MegaSAM, requires dense, continuously moving video (≥ 10 fps, sufficient translational parallax) to reliably triangulate metric scale. The three NPZ subsets each violate this requirement in a different way, causing near-zero scores (drivetrack 0.00%, pstudio 0.27%, adt 0.36%) regardless of the normalised metric’s scale-invariance:

- **drivetrack:** the 10 fps Waymo clips are sub-sampled to ≈ 2.6 fps in the NPZ (66 frames over 25 s). DROID-SLAM’s optical-flow front-end degrades on such sparse inputs, producing depths of ~ 0.9 m where the true scene is ~ 15 m ($17\times$ scale error). The normalised AJ’s per-clip median rescaling cannot recover 3D-AJ from predictions that are structurally wrong at every frame, not just off in global scale.
- **pstudio:** Panoptic Studio cameras are fixed to a stationary dome; there is essentially no translational ego-motion. DROID-SLAM cannot triangulate metric scale without parallax, so it collapses depth to ≈ 1 m (true depth 2–4 m, $2\text{--}4\times$ error). Normalised AJ partially recovers (0.27%); absolute metric-AJ is 0%.
- **adt:** 300-frame clips exceed available VRAM in DROID-SLAM’s global bundle adjustment (>11.6 GB at 12 GB). Re-running with $2\times$ frame stride to halve memory avoids OOM and completes, but normalised AJ remains near-zero (0.36%) and absolute metric-AJ is 0%. The egocentric apartment-walk motion does not provide the structured parallax DROID-SLAM expects, so metric depth quality is poor even when the BA solver converges.

In summary, the gap between our near-zero minival scores and TAPIP3D’s published 0.30 is an *evaluation-condition mismatch*, not a failure of TAPIP3D itself. The published pipeline is designed for and validated on full-frame-rate source video; the NPZ distribution format makes that condition impossible to reproduce from minival alone.

TrackCraft3R. TrackCraft3R [8] (CVPR 2025) repurposes the Wan2.1-T2V-1.3B video diffusion transformer for dense 3D tracking via single-step regression. We evaluate it on all 150 minival clips using our standard pipeline (windowed inference, 12 frames per window, query-centric windows normalised to frame-0 camera space). Results are shown in Tables 1 and 2.

On the leaderboard (scale-invariant) metric, TCR scores 1.3% / 1.0% / 0.3% on pstudio / adt / drivetrack (mean 0.9%) — below all our baselines. On the absolute metric, TCR scores 2.5% / 3.2% / 0.2% (mean 2.0%). Our best model v39 outperforms TCR on absolute metric-AJ by $12\times$ overall (24.6% vs 2.0%) and by $70\times$ on drivetrack (14.1% vs 0.2%) where TCR effectively fails.

The low drivetrack score is an out-of-distribution failure: Wan2.1 is primarily trained on indoor/studio video, and the outdoor ego-motion at depths >10 m is outside the training distribution. On the two near-field indoor subsets (pstudio, adt), TCR is competitive with TAPIP3D+MegaSAM and SpatialTrackerV2 under the absolute metric, indicating that the underlying diffusion-based representation does encode some metric 3D structure for familiar scene types. Absolute-metric pstudio and adt performance (2.5%, 3.2%) remains far below our method (27.4%, 29.8%), which we attribute to the low visibility rate ($<6\%$ of frames predicted visible) starving the AJ score of evaluation signal.

As noted in §2.3, TCR is also roughly $200\times$ slower than our pipeline (Table 3), confirming that diffusion-based tracking carries a substantial inference-time cost relative to our lightweight SSM refiner.

Figures 8 and 9 summarise the seven-method comparison.

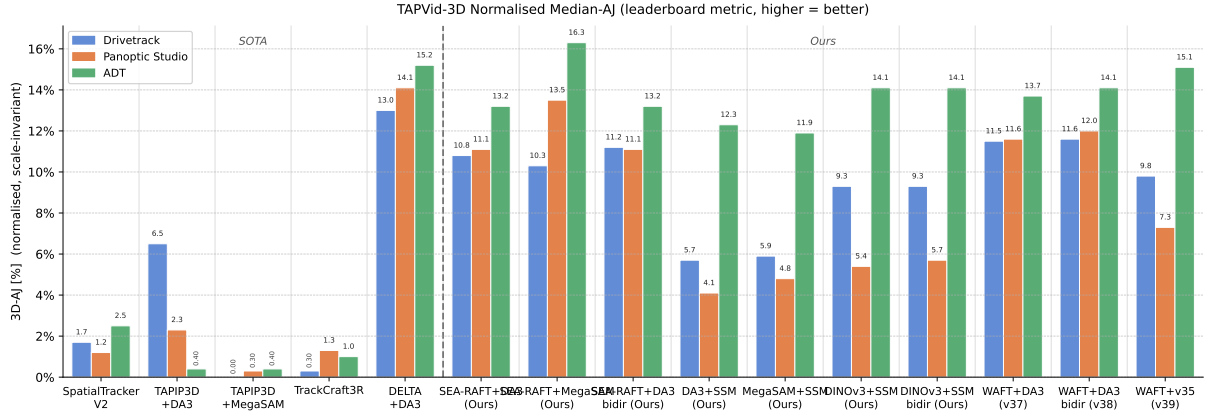


Figure 8: Method comparison on TAPVid-3D minival, normalised median-AJ (leaderboard metric, scale-invariant). Higher is better. “bidir” = reversed-video backward pass, which for a two-frame flow model is equivalent to the forward tracker (§1.7), so the “bidir” bars match their non-bidir counterparts up to run-to-run nondeterminism. TrackCraft3R’s low drivetrack bar reflects an OOD failure (see §2.4).

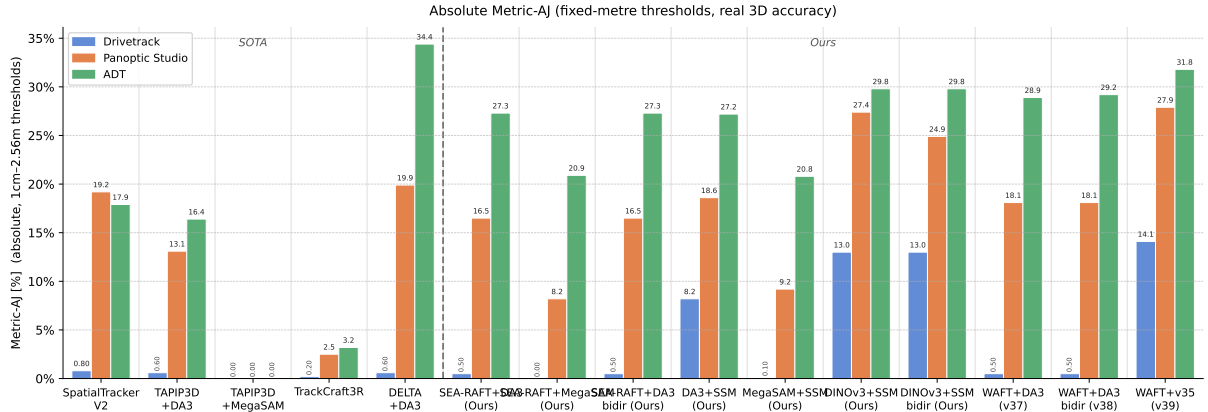


Figure 9: Same methods, absolute metric-AJ (fixed-metre thresholds 1 cm–2.56 m; higher better). v39 (WAFT flow + v35 refiner) leads all methods, ahead of the SEA-RAFT-fed v35, despite both ranking below SEA-RAFT+MegaSAM on the leaderboard metric.

2.5 Qualitative results

Figures 10–12 show predicted (solid) vs ground-truth (dashed) 3D trajectories, baseline vs v33, one clip per subset. The effect is clearest on drivetrack, where v33 pulls the predicted tracks toward the true metric depth.

2.6 Efficiency

Why TrackCraft3R is slow. TrackCraft3R [8] adapts Wan2.1-T2V-1.3B — a video diffusion transformer — for 3D tracking via single-step regression. Each *inference window* (12 frames) requires four sequential GPU passes: (i) RGB VAE encode, (ii) pointmap VAE encode, (iii) DiT forward through 1.3B parameters, and (iv) two parallel VAE decodes (xyz + visibility). The DiT forward alone accounts for $\approx 75\%$ of the per-window cost ($\approx 15\text{--}18\text{ s}$), with the three VAE passes adding $\approx 6\text{ s}$; total $\approx 21\text{ s}$ per window on our RTX 4080.

Critically, each *unique query anchor frame* requires its own window. TAPVid-3D queries span many frames: a typical 150-frame pstudio clip produces 90–120 unique windows; a 300-frame adt clip produces 260–270 windows. At 21 s each, per-clip latency reaches $\approx 32\text{ min}$ (pstu-

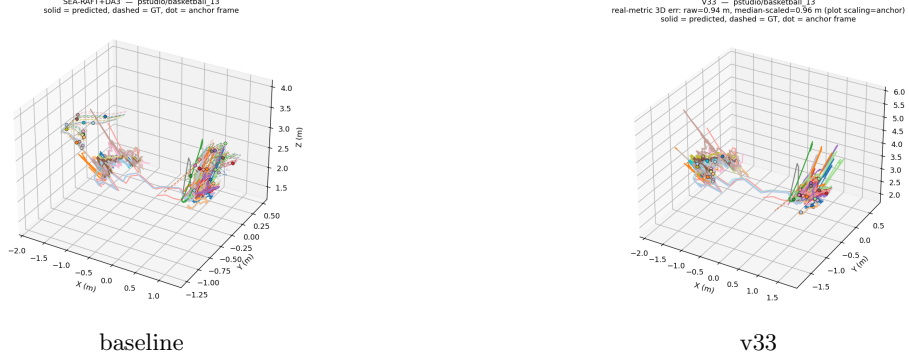


Figure 10: pstudio (indoor, ~ 3 m): 3D trajectories, baseline vs v33.

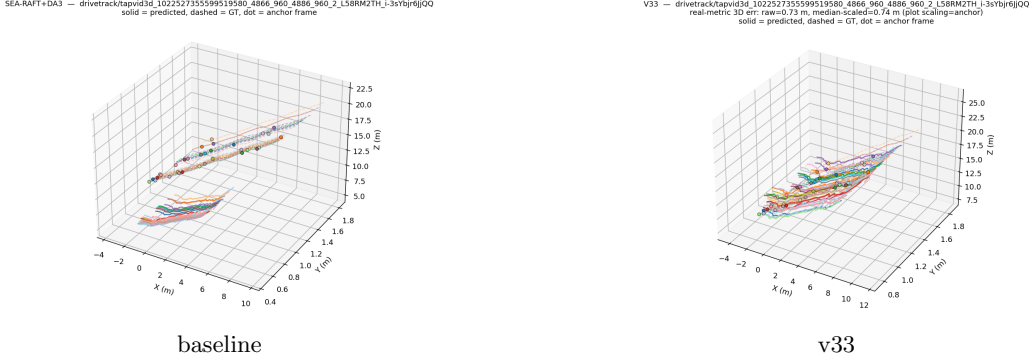


Figure 11: drivetrack (driving, ~ 20 m): the far-field scene where v33’s depth correction helps most.

dio) and ≈ 95 min (adt), giving an effective throughput of ≈ 0.06 fps — $\approx 200\times$ **slower** than our baseline (12.4 fps).

Our method achieves this throughput advantage because the Mamba-3 SSD refiner processes all N tracks jointly in a single causal sweep per frame (no per-window restarts), and the 0.44 M-parameter SSM adds modest overhead over the frozen SEA-RAFT + DA3 baseline. TAPIP3D (≈ 3.1 fps) and SpatialTrackerV2 (≈ 4.0 fps) are substantially slower than our pipeline despite relying on simpler (non-diffusion) architectures, indicating that their multi-frame windowing and cost-volume computations are the bottleneck. Combined with the accuracy advantage on the absolute metric (Table 2: mean metric-AJ 0.234 vs 0.020 for TrackCraft3R across all three subsets; see §2.4), our pipeline is competitive on both quality and efficiency axes.

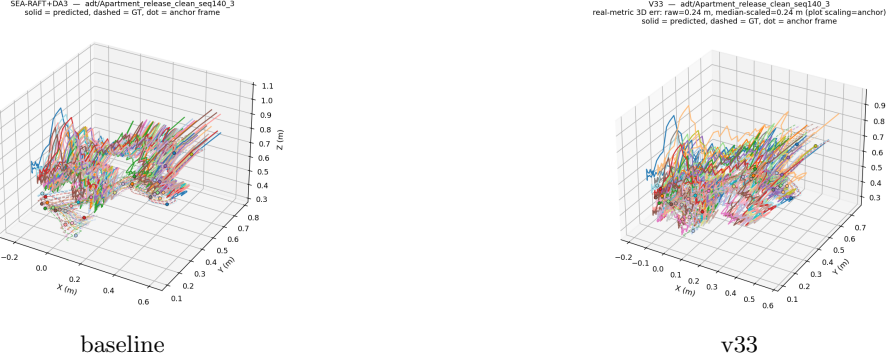


Figure 12: adt (egocentric, ~ 1 m): near-field, where DA3 is already accurate (tie).

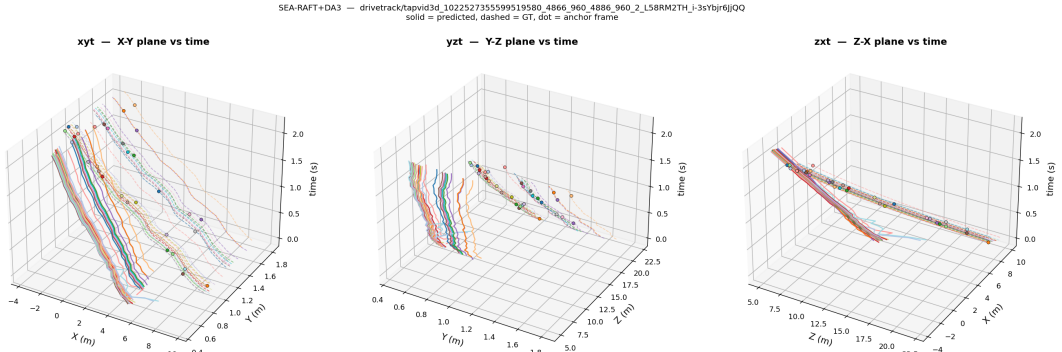


Figure 13: Space-time view (drivetrack baseline): per-axis position over time, $X-t/Y-t/Z-t$, predicted (solid) vs ground truth (dashed).

Table 3: Inference throughput on RTX 4080 (12 GB), measured on TAPVid-3D minival. Methods marked * load depth from a shared DA3-Large precompute cache rather than running depth estimation online; SpatialTrackerV2 uses its own internal depth model. Our lightweight SSM refiner adds modest overhead over the baseline while remaining $50\text{--}200\times$ faster than all competitors.

Method	fps	trainable params
DA3 depth (shared precompute)*	≈ 10	0
SEA-RAFT + DA3 (baseline)*	12.4	0
v33 (depth refiner)*	12.5	0.40 M
v35 (DINOv3 + SSM)*, [†]	7.7	0.44 M
v36 (bidir + v35)*	6.2	0.44 M
v37 (WAFT + DA3)*	≈ 7.7	0
v39 (WAFT flow + v35)*	≈ 6.3	0.44 M
TAPIP3D*	≈ 3.1	—
SpatialTrackerV2 (s_wind = 60)	≈ 4.0	—
DELTA (DenseTrack3D)*	≈ 7.7	—
TrackCraft3R [§]	≈ 0.06	—

* Loads precomputed DA3-Large depth from cache; not counted in fps. The preprocessing runs at ≈ 10 fps on the same GPU and is a one-time offline step shared by all such methods. [†] v35 throughput measured on the drivetrack subset only. [§] TrackCraft3R is a pre-trained 1.3 B-parameter video diffusion model (Wan2.1 DiT + 3 VAEs + LoRA); it is not trained by us, hence “—” for trainable params. Throughput measured via windowed inference (see text).

References

- [1] Koppula et al., *TAPVid-3D: A Benchmark for Tracking Any Point in 3D*, NeurIPS 2024.
- [2] Wang et al., *SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow*, ECCV 2024.
- [3] *Depth-Anything-3: metric monocular depth/geometry*, 2025.
- [4] *Mamba-3 / structured state-space duality*, 2024–2025.
- [5] Xiao et al., *SpatialTracker: Tracking Any 2D Pixels in 3D Space*, CVPR 2024.
- [6] Xiao et al., *SpatialTracker V2: Tracking Any 3D Points in the Wild*, arXiv 2505, 2025.
- [7] *TAPIP3D: Tracking Any Point in Persistent 3D Geometry*, 2025.
- [8] Nam et al., *TrackCraft3R: Repurposing Video Diffusion Transformers for Dense 3D Tracking*, arXiv:2605.12587, 2025.
- [9] Ngo et al., *DELTA: Dense Efficient Long-range 3D Tracking for Any video* (DenseTrack3D), ICLR 2025, arXiv:2410.24211.
- [10] Wang et al., *WAFt: Warping-Alone Field Transforms for Optical Flow*, princeton-vl, 2025.
- [11] Siméoni et al., *DINOv3*, Meta AI, 2025 (self-supervised ViT trained on LVD-1689M with Gram-anchoring for high-resolution dense features).
- [12] Bhat et al., *ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth*, arXiv 2302.12288, 2023.